

# UNIVERSIDAD RAFAEL LANDÍVAR

ÁREA DE INGENIERÍA

Programación Web



## Ejercicio 1

API Rest

AUTOR

Sajvin Gómez, Mario Daniel (1612921)

DOCENTE

Inge. Ricardo Trujillo

/suma

```
// sumar GET
app.get('/suma', (req, res) => {
  const a = parseFloat(req.query.a);
  const b = parseFloat(req.query.b);
  res.send({
    SumaGet: a+b,
  });
})
```

GET http://localhost:3000/suma?a=99&b=6 Send

Status: 200 OK Size: 15 Bytes Time: 26 ms

Query Headers Auth Body Tests Pre Run

Query Parameters

|                                     |           |       |   |
|-------------------------------------|-----------|-------|---|
| <input checked="" type="checkbox"/> | a         | 99    | ▼ |
| <input checked="" type="checkbox"/> | b         | 6     | ▼ |
| <input type="checkbox"/>            | parameter | value |   |

Response Headers Cookies Results Docs

```
1 {
2   "SumaGet": 105
3 }
```

Copy

```
// sumar POST
app.post('/suma', (req, res) => {
  const {a, b} = (req.body);
  resultado = parseFloat(a) + parseFloat(b)
  res.send({
    SumaPost: resultado,
  })
})
```

POST http://localhost:3000/suma Send

Status: 200 OK Size: 15 Bytes Time: 5 ms

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "a": 10,
3   "b": 5
4 }
```

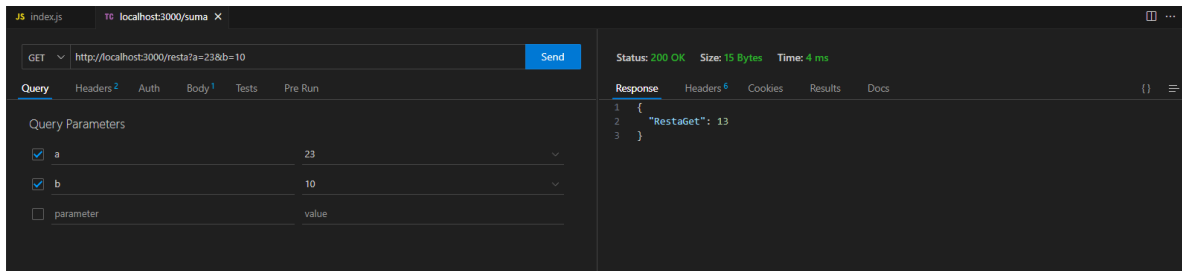
Format

Response Headers Cookies Results Docs

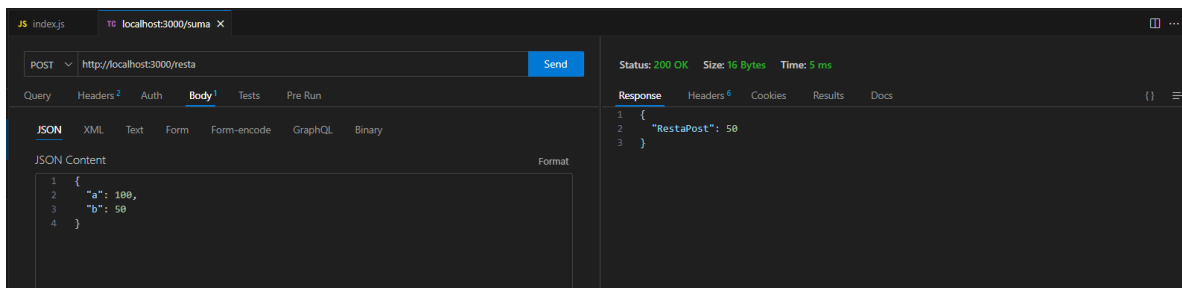
```
1 {
2   "SumaPost": 15
3 }
```

/resta

```
// restar GET
app.get('/resta', (req, res) => {
  const a = parseFloat(req.query.a);
  const b = parseFloat(req.query.b);
  res.send({
    RestaGet: a-b,
  });
});
```

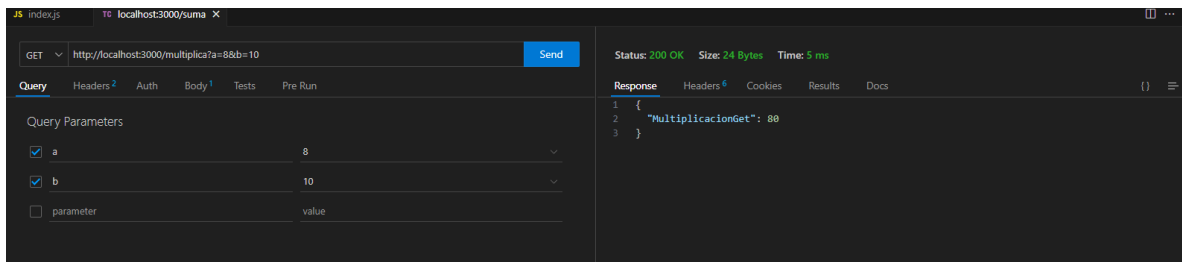


```
// restar POST
app.post('/resta', (req, res) => {
  const {a, b} = (req.body);
  resultado = parseFloat(a) - parseFloat(b)
  res.send({
    RestaPost: resultado,
  })
});
```



/multiplicación

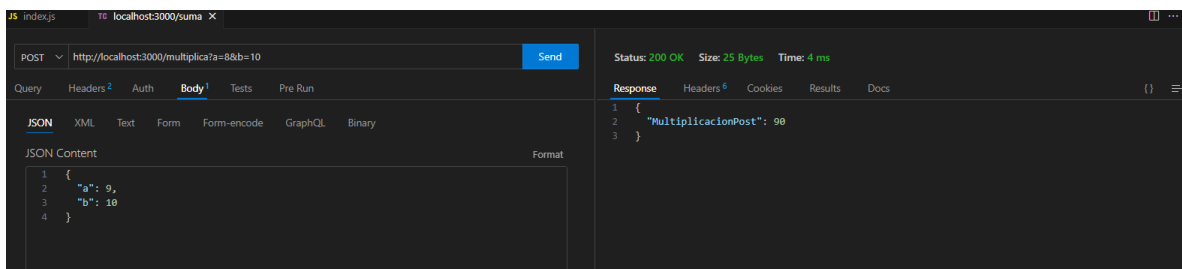
```
// multiplicacion GET
app.get('/multiplica', (req, res) => {
  const a = parseFloat(req.query.a);
  const b = parseFloat(req.query.b);
  res.send({
    MultiplicacionGet: a*b,
  });
});
```



The screenshot shows a web browser window with the address bar displaying `http://localhost:3000/multiplica?a=8&b=10`. The browser's developer tools are open, showing the 'Query' tab. The 'Query Parameters' section lists `a=8` and `b=10`. The 'Response' tab shows the following JSON object:

```
{
  "MultiplicacionGet": 80
}
```

```
// multiplicacion POST
app.post('/multiplica', (req, res) => {
  const {a, b} = (req.body);
  resultado = parseFloat(a) * parseFloat(b)
  res.send({
    MultiplicacionPost: resultado,
  })
});
```



The screenshot shows a web browser window with the address bar displaying `http://localhost:3000/multiplica?a=8&b=10`. The browser's developer tools are open, showing the 'Body' tab. The 'JSON Content' section shows the following JSON object:

```
{
  "a": 9,
  "b": 10
}
```

The 'Response' tab shows the following JSON object:

```
{
  "MultiplicacionPost": 90
}
```

/división

```
// division GET
app.get('/dividir', (req, res) => {
  const a = parseFloat(req.query.a);
  const b = parseFloat(req.query.b);
  res.send({
    DivisionGet: a/b,
  });
});
```

The screenshot shows a web client interface with a GET request to `http://localhost:3000/dividir?a=80&b=10`. The query parameters are `a=80` and `b=10`. The response is a JSON object: `{\"DivisionGet\": 8}`.

```
// division POST
app.post('/dividir', (req, res) => {
  const {a, b} = (req.body);
  resultado = parseFloat(a) / parseFloat(b)
  res.send({
    DivisionPost: resultado,
  })
});
```

The screenshot shows a web client interface with a POST request to `http://localhost:3000/dividir`. The body is a JSON object: `{\"a\": 5, \"b\": 2}`. The response is a JSON object: `{\"DivisionPost\": 2.5}`.